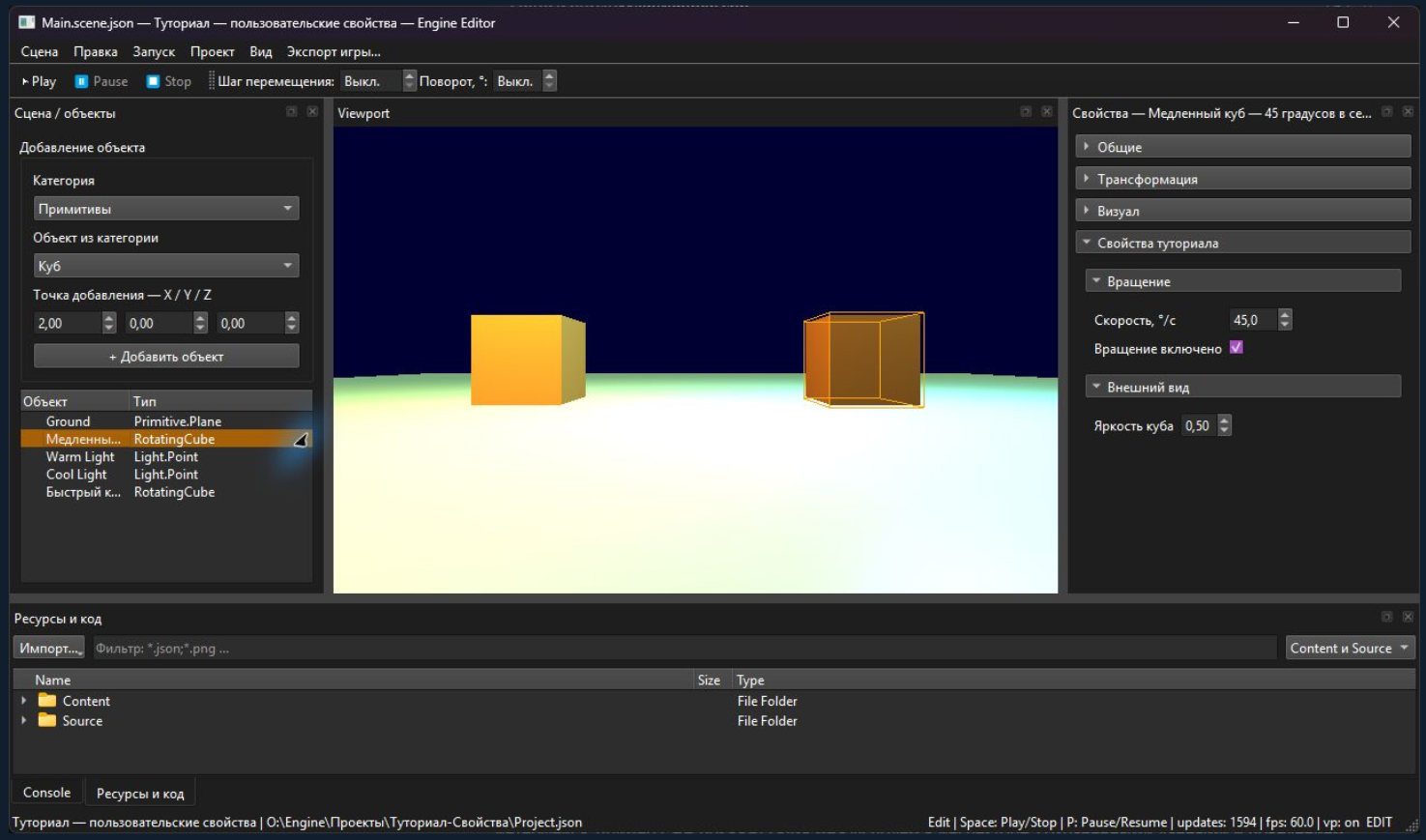
PainEngine Практический туториал

Редактор, игровой UI
и рабочие примеры C++



2026.10.31.2 / 31.10.2026

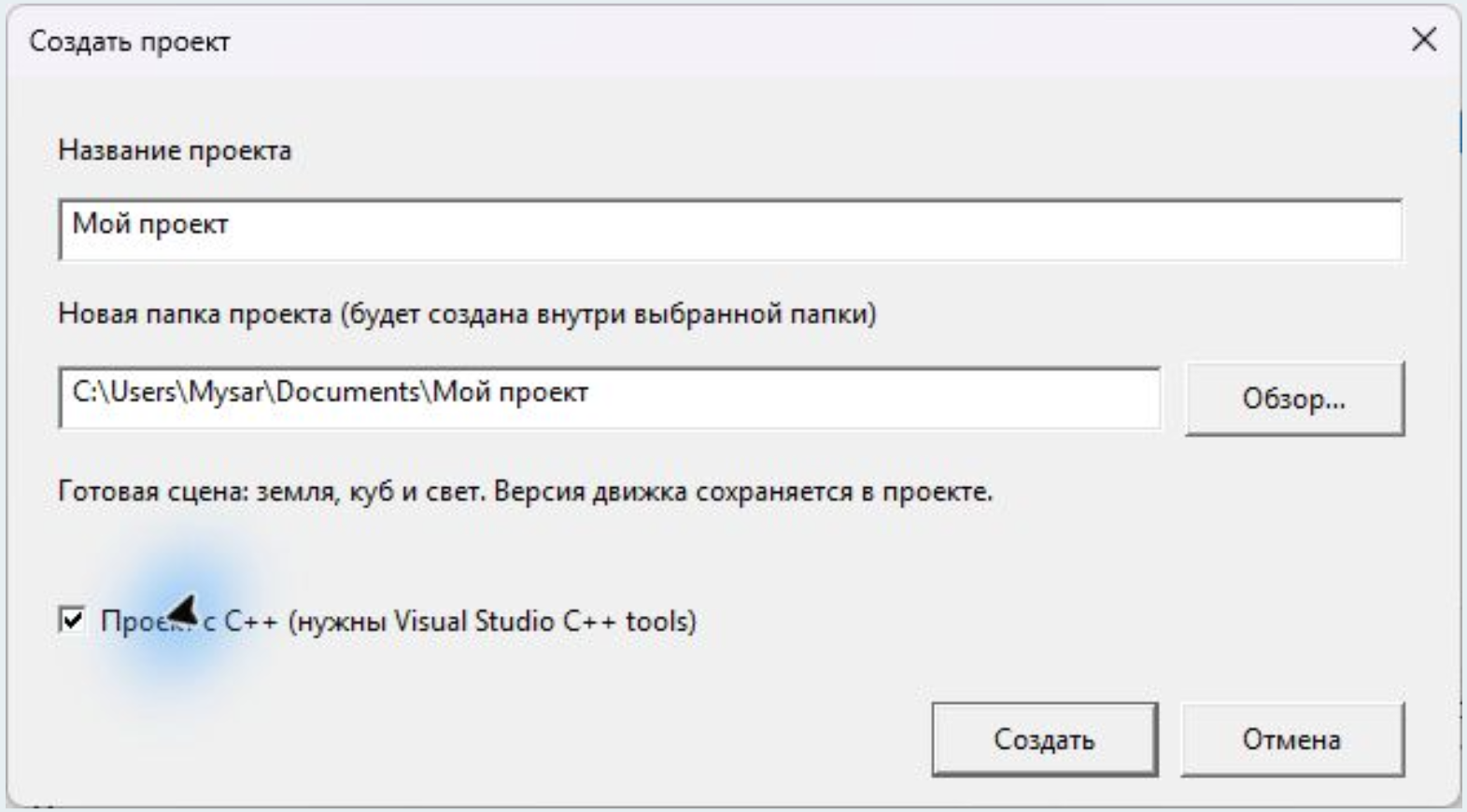
Первый проект в лаунчере

Создать проект

Выберите папку и название. C++ можно включить сразу или использовать шаблон без кода.

Редактировать

Выберите созданный проект в списке лаунчера и откройте редактор.



Флажок C++ создаёт исходники, включая RotatingCube. Нужны Visual Studio C++ tools и Windows SDK.

Окно редактора

Слева

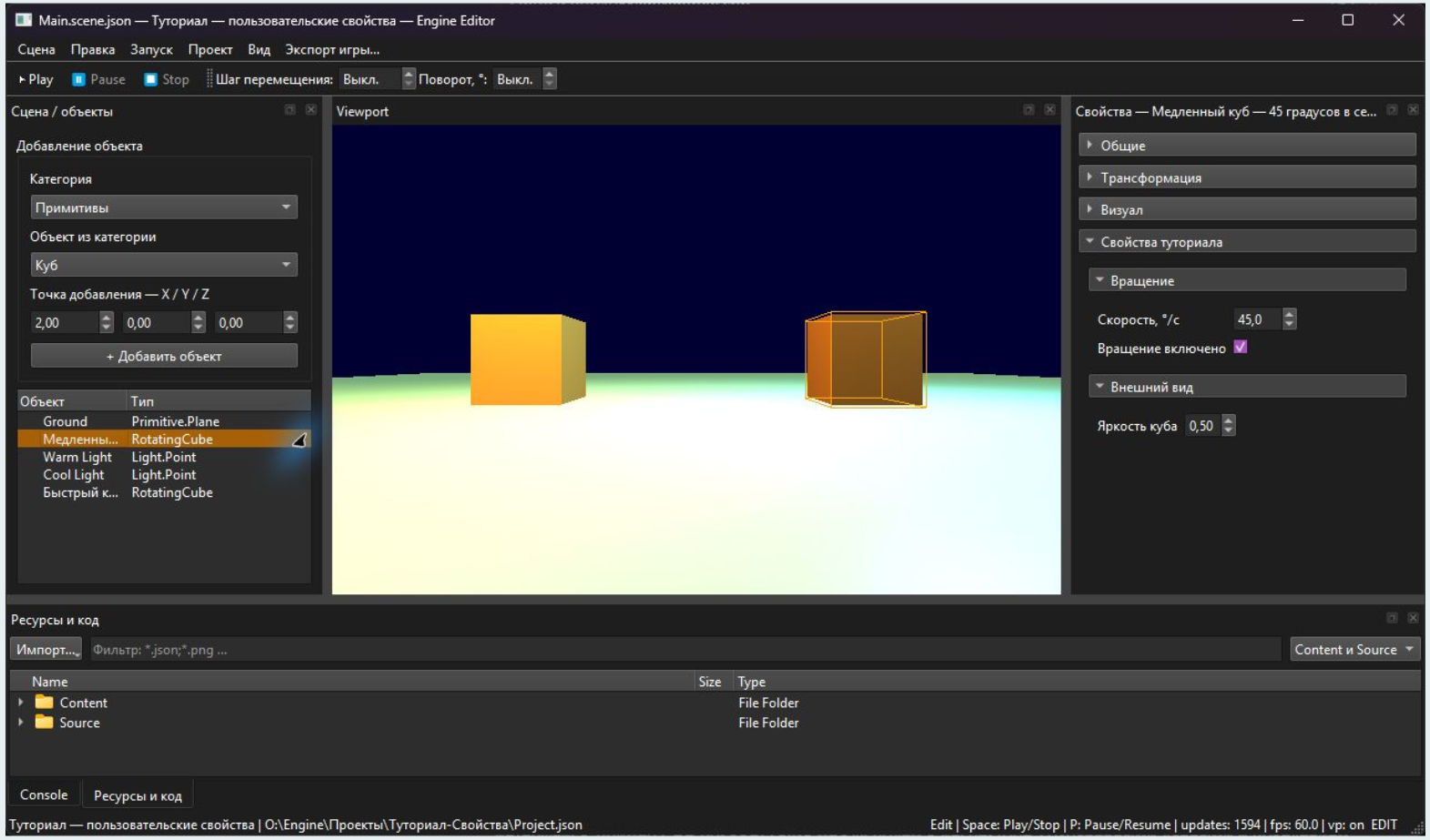
Каталог типов и объекты сцены. Выберите куб в дереве.

Справа

Сворачиваемые разделы и поля выбранного объекта.

Снизу

Ресурсы и код; рядом вкладка Console. Высоту можно менять.



Панель контента: высота и положение

Изменить высоту

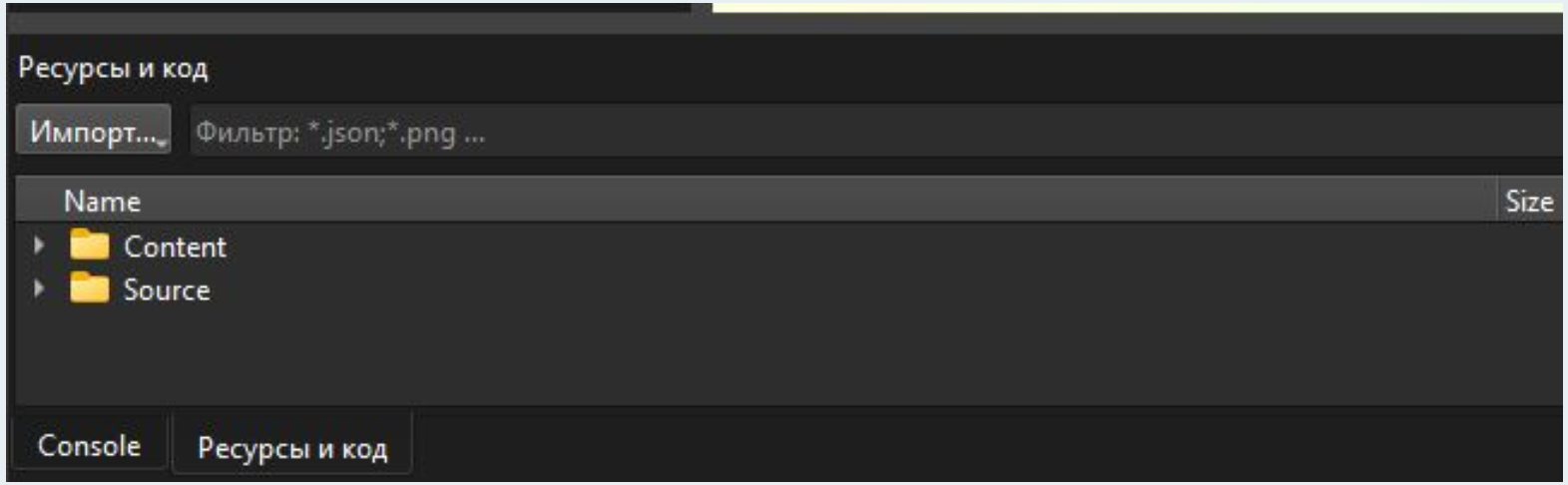
Тяните широкую серую границу над «Ресурсы и код».

Переместить

Тяните заголовок или нижнюю вкладку панели.

Запомнить

Раскладка сохраняется отдельно для каждого проекта.



Граница меняет размер; заголовок и вкладка меняют место панели.

Вернуть удобную раскладку

Компактная панель

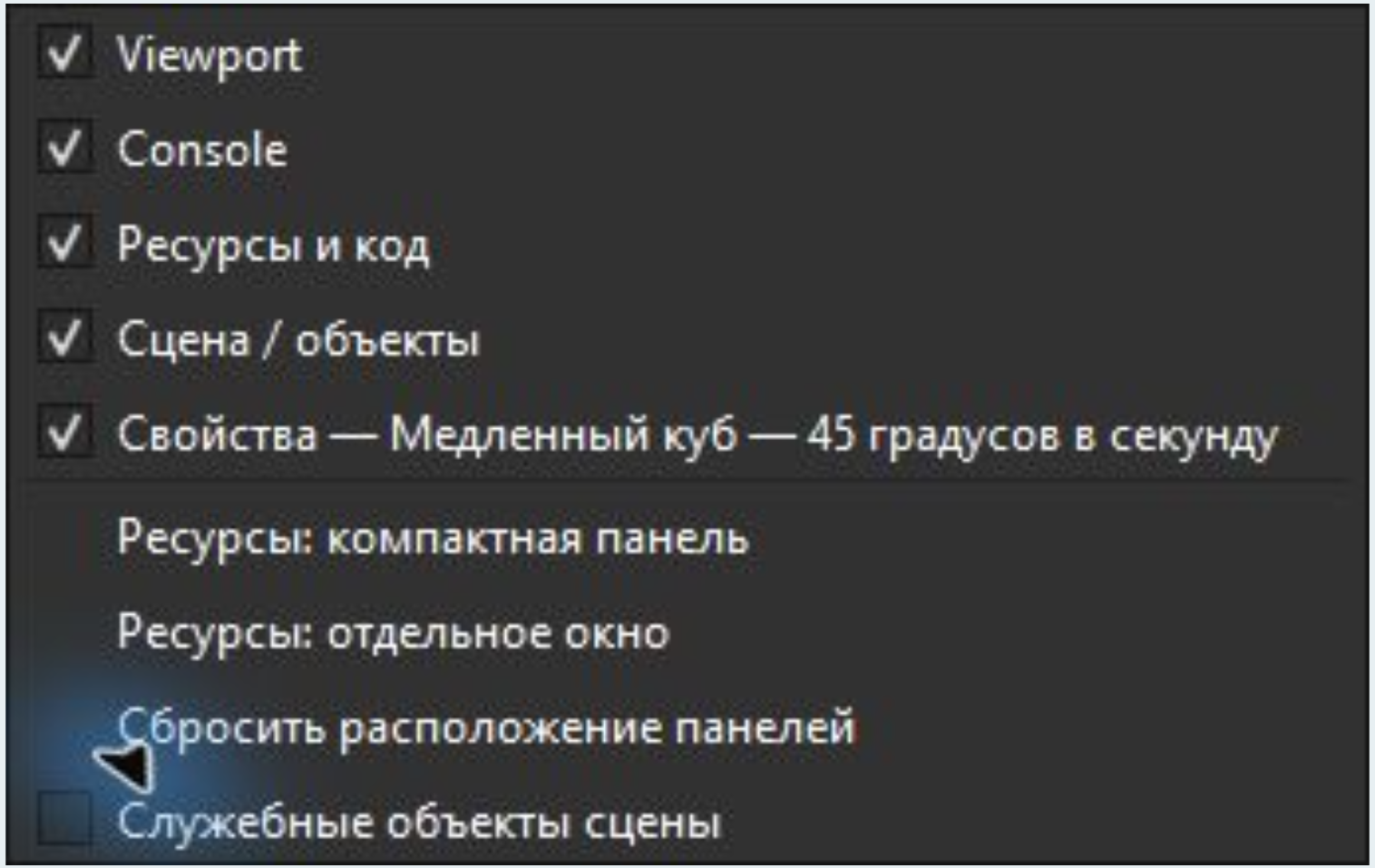
Вид → Ресурсы: компактная панель. Освобождает место сцене.

Отдельное окно

Вид → Ресурсы: отдельное окно. Панель можно вынести.

Начать заново

Вид → Сбросить расположение панелей. Возвращает исходную схему.



Закрытую панель включите снова в меню «Вид».

Камера и выделение в Edit

Камера

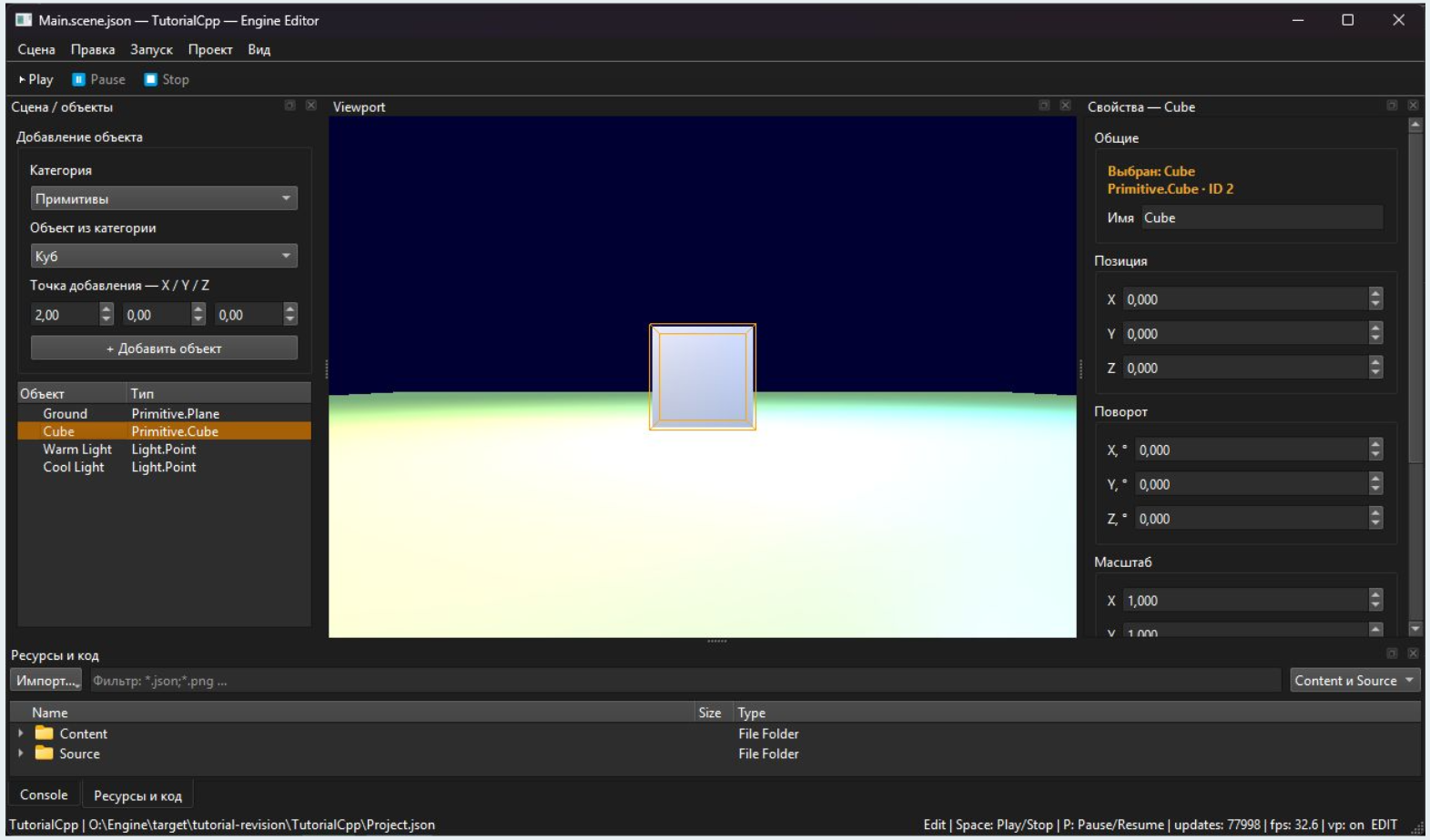
WASD; E / Q вверх и вниз. ПКМ + мышь для обзора, Shift для ускорения.

Выделение

ЛКМ — выбор; Tab — перебор; F — фокус; Esc — снять выбор.

Перемещение

Стрелки и Page Up / Page Down. Точные значения вводятся в свойствах.



Сначала щёлкните viewport. При вводе в текстовое поле горячие клавиши сцены не действуют.

Несколько объектов и привязки

Выбрать группу

Ctrl / Shift + щелчок в дереве или viewport. Трансформации применяются ко всей группе.

Переместить точно

Задайте шаг перемещения и угол на верхней панели. Ноль отключает привязку.

Отменить целиком

Ctrl+D, удаление и перетаскивание работают как одна операция Undo.

Остальные свойства относятся к активному объекту. Нужна актуальная DLL проекта.

Импорт модели и текстуры

Импорт

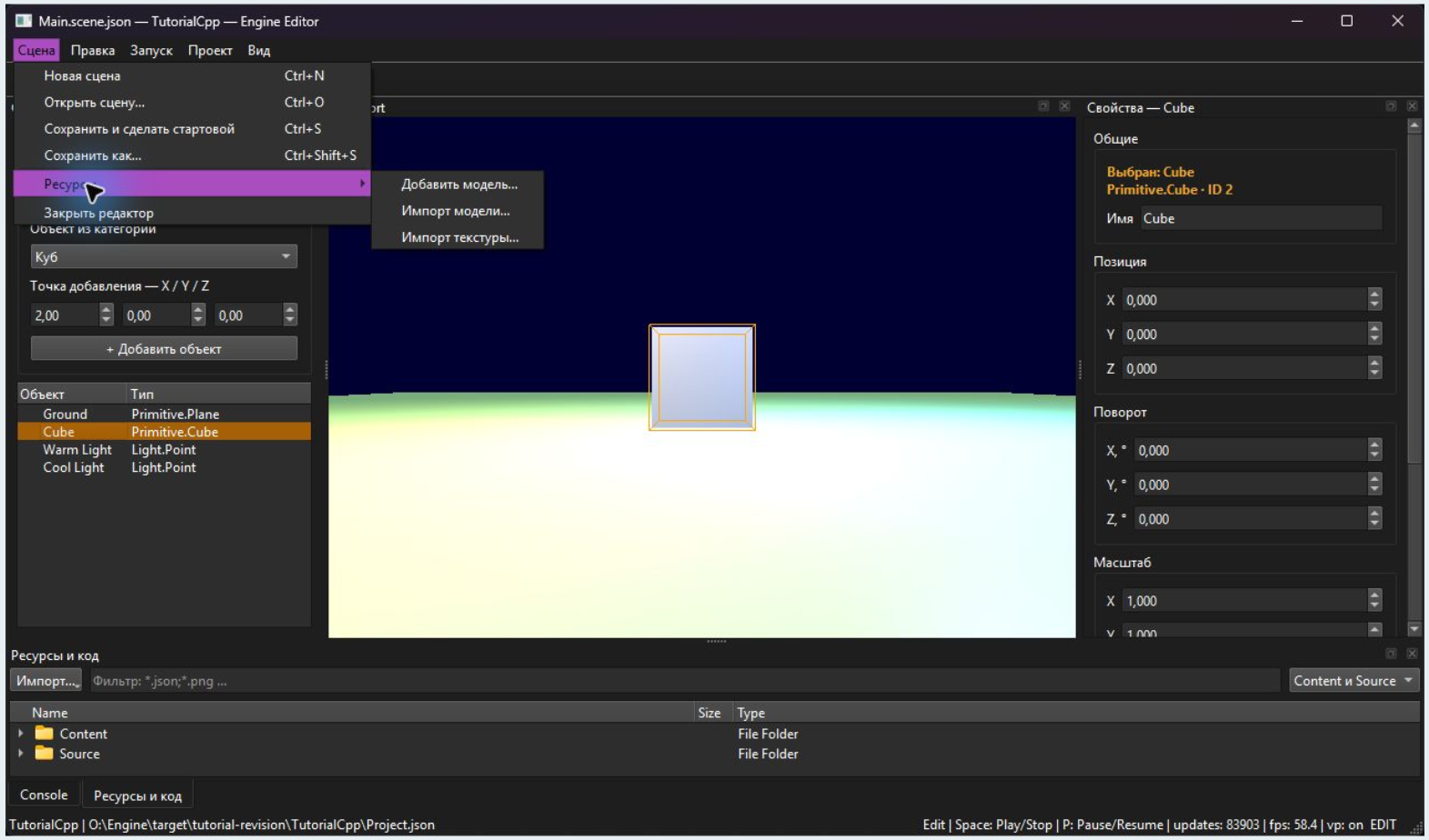
Сцена → Ресурсы → Импорт модели… или Импорт текстуры…

Добавление

Дважды щёлкните модель в Content/Models, чтобы добавить её в сцену.

Назначение

Для отдельной текстуры: выберите объект и нажмите «Выбрать текстуру…».



Модели: GLB, glTF, OBJ, FBX, BLEND. Изображения: PNG, JPG, BMP, TGA.

FBX / BLEND: модель сразу с материалом

Что создаётся

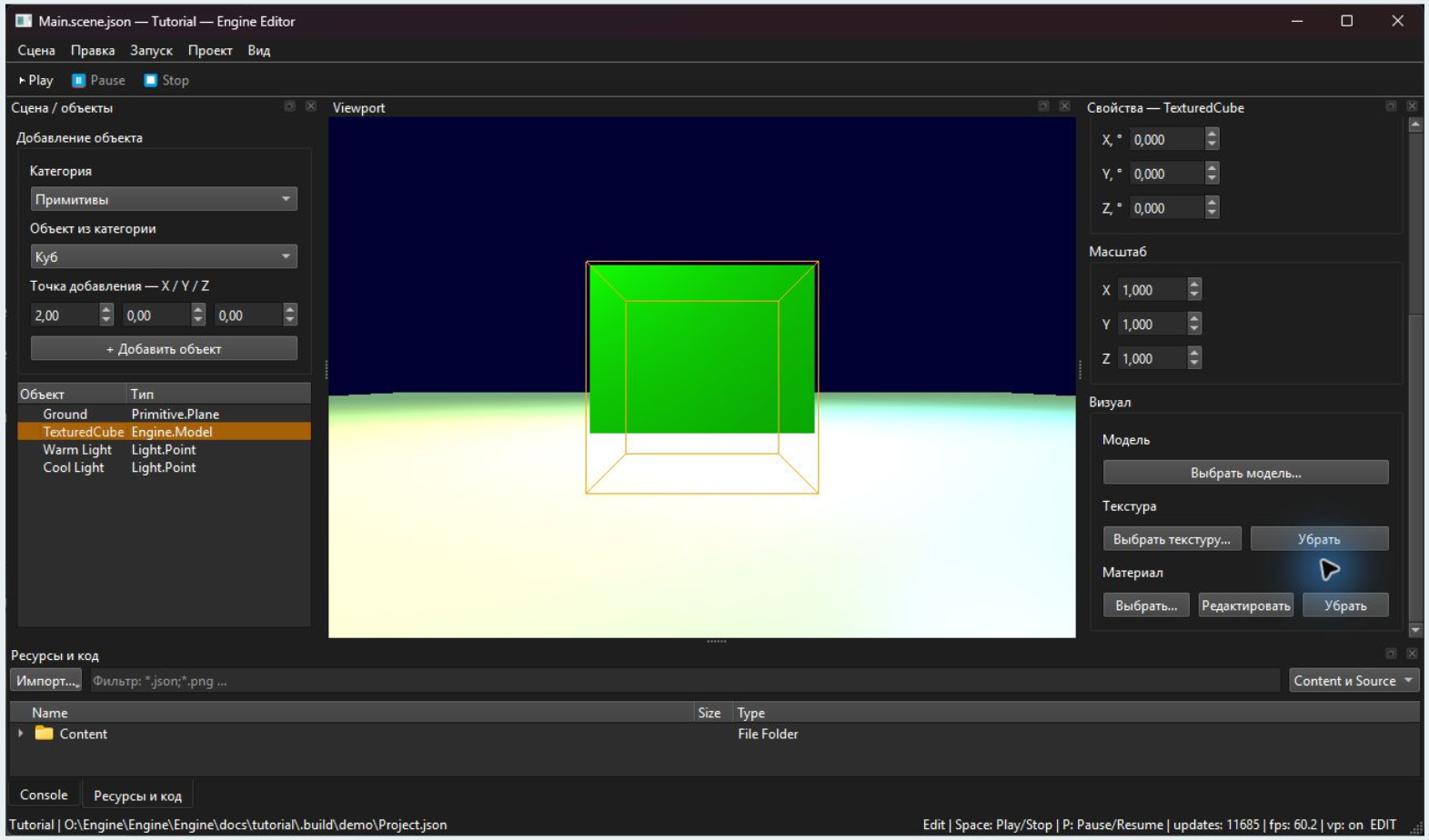
Модель, папки Materials и Textures, файл .import.json.

При добавлении

Материал назначится сам. BaseMap — текстура, Tint — базовый цвет.

Blender

Blender 4.2+. Переносится Image Texture из Base Color.



Базовый opaque-материал и UV0. Normal, roughness, metallic и сложный граф узлов не переносятся.

Обновить ресурс, сохранив сцену

Переимпортировать	ПКМ по модели или текстуре → Переимпортировать… Укажите исходный файл.
Перенести	ПКМ → Переместить / переименовать… Задайте новый путь внутри Content.
Проверить и сохранить	Посмотрите сцену и сохраните её. История Undo очищается; файлы остаются в резервной копии.

Модель переносится всей папкой; имя главного файла сохраняется. Операции доступны в Edit.

Редактор материала

Открыть

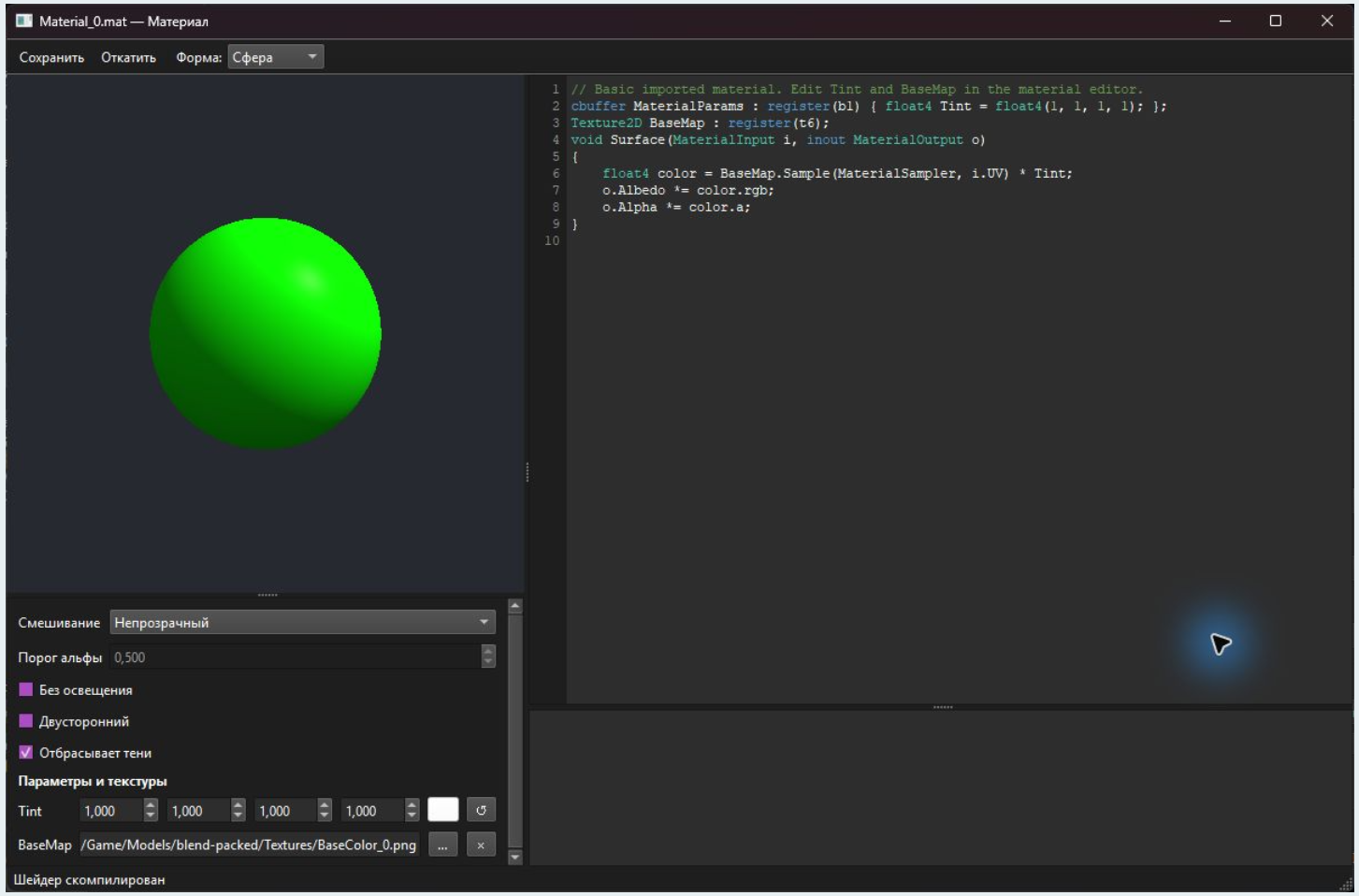
Свойства → Материал → Редактировать или двойной щелчок по .mat.

Настроить

Слева — превью, Tint и BaseMap. Справа — код HLSL поверхности.

Сохранить

Параметры видны сразу. Ctrl+S в этом окне сохраняет .mat и применяет HLSL.



Ctrl+S основного окна сохраняет сцену. F6 перечитывает материалы после внешних изменений.

Направленный свет и тени

Выбрать или добавить

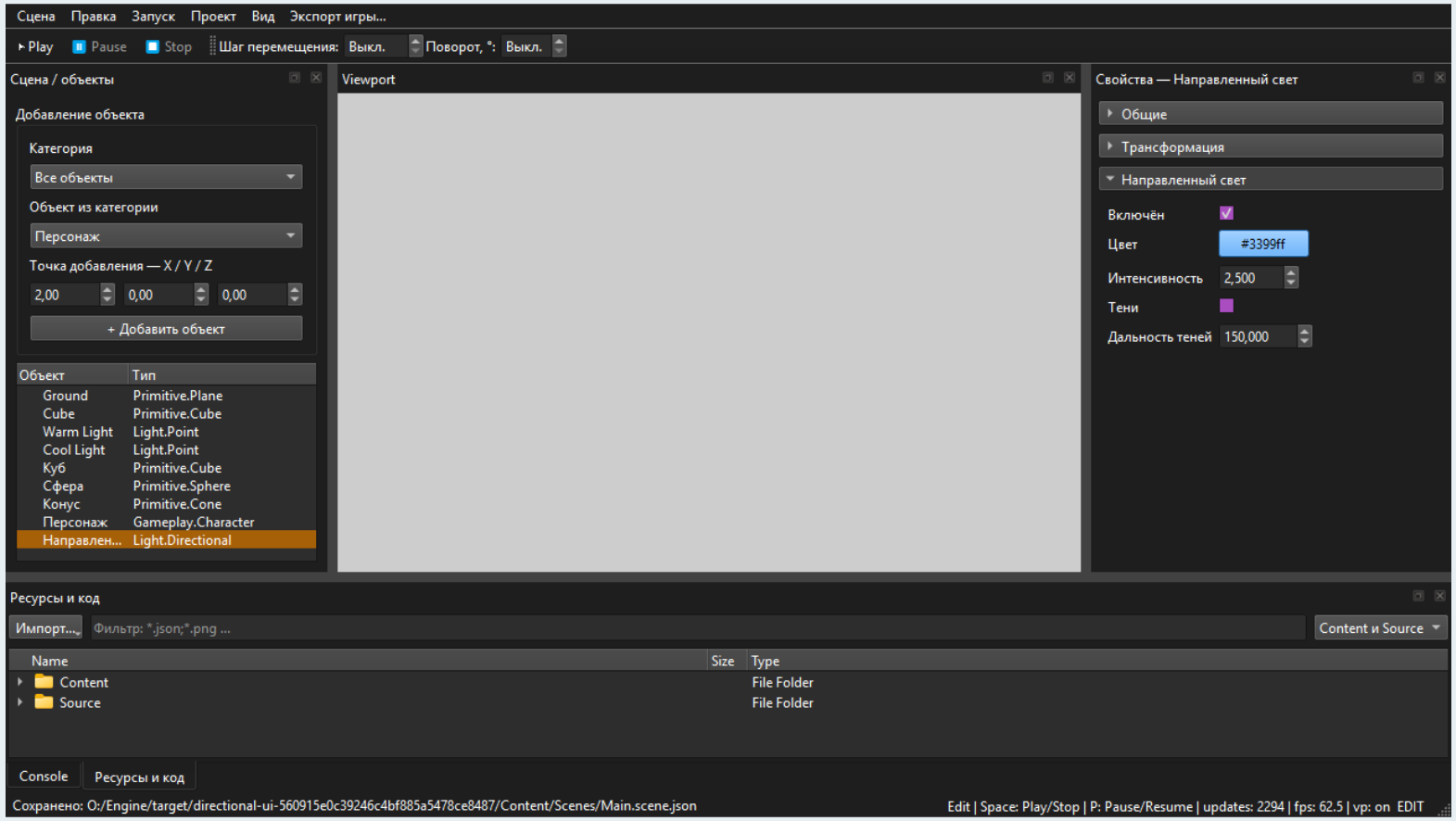
Сцена → Направленный свет. В сцене может быть один такой источник.

Повернуть

Поверните источник в трансформации. Позиция и масштаб не меняют свет.

Настроить

Задайте цвет, интенсивность, флажок и дальность теней.



Снимок показывает реальные панели. D3D-изображение viewport в этот захват не попало.

Точечный свет, частицы и небо

Точечный свет

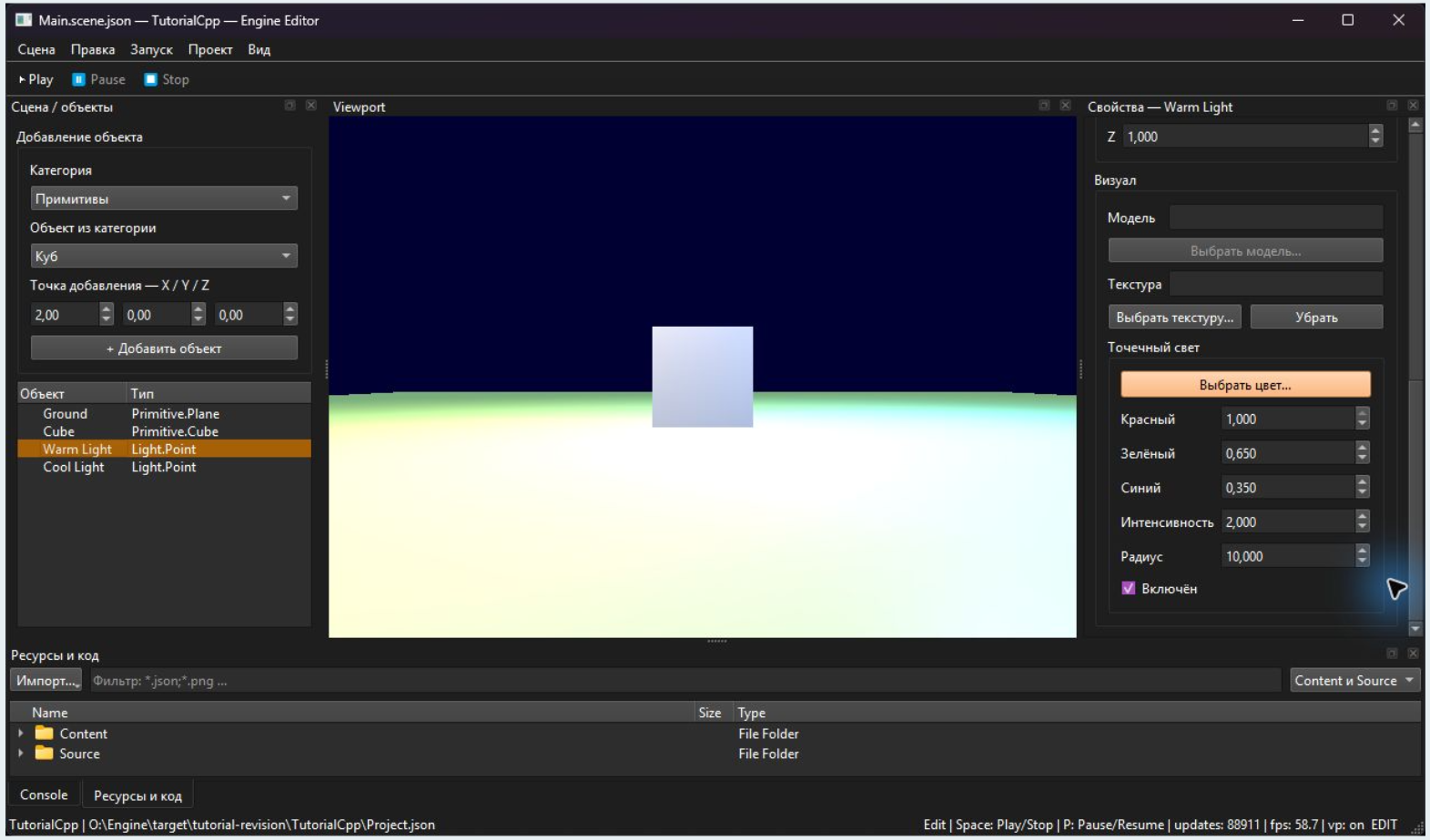
Выберите точечный источник. Настройте цвет, интенсивность и радиус.

Частицы

В категории «Частицы» выберите искры или дым.

Небо

Категория «Окружение» → «Базовое небо».



Параметры выбранного типа находятся ниже трансформации в правой панели.

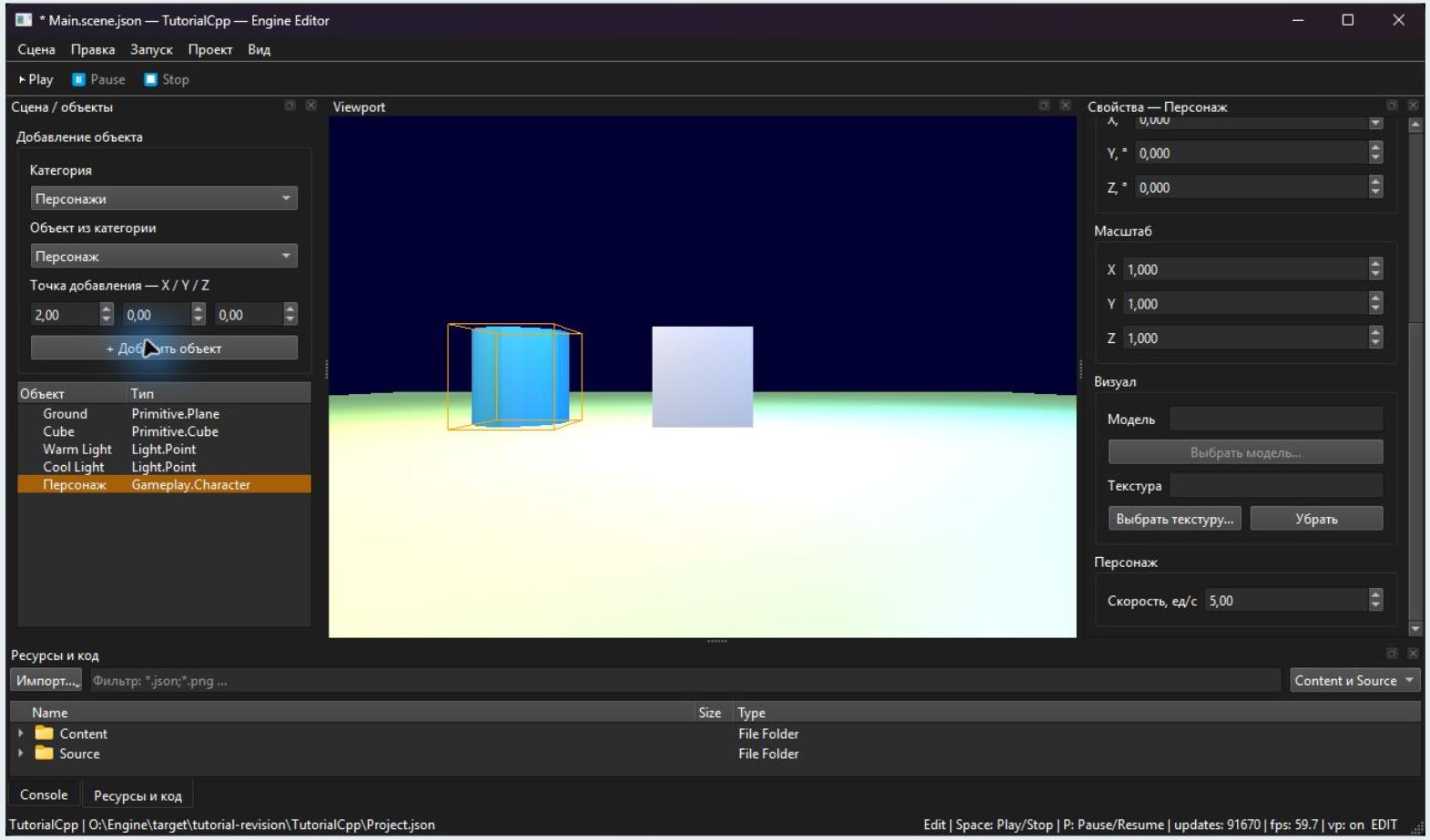
Стандартный персонаж

Добавление

Категория «Персонажи» → «Персонаж» → «Добавить объект».

Управление

Назначьте персонажа в настройках Play.



Назначение для Play

Персонаж

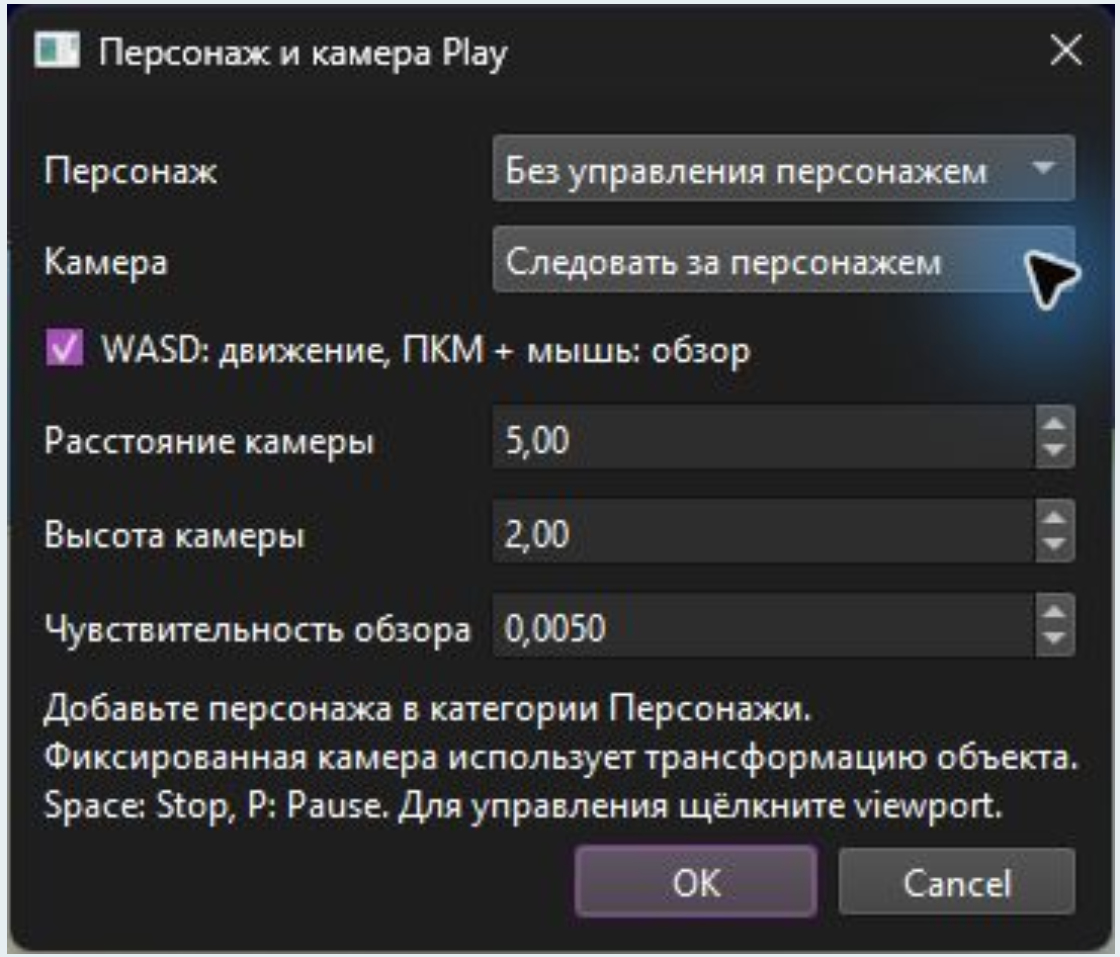
Выберите объект сцены в первом поле.

Камера

«Следовать за персонажем» или отдельная «Камера Play».

Ввод

Включите WASD и обзор с ПКМ. Затем щёлкните viewport в Play.



Запуск → Настроить Play…

Edit, Play, Pause и Stop

Play / Pause

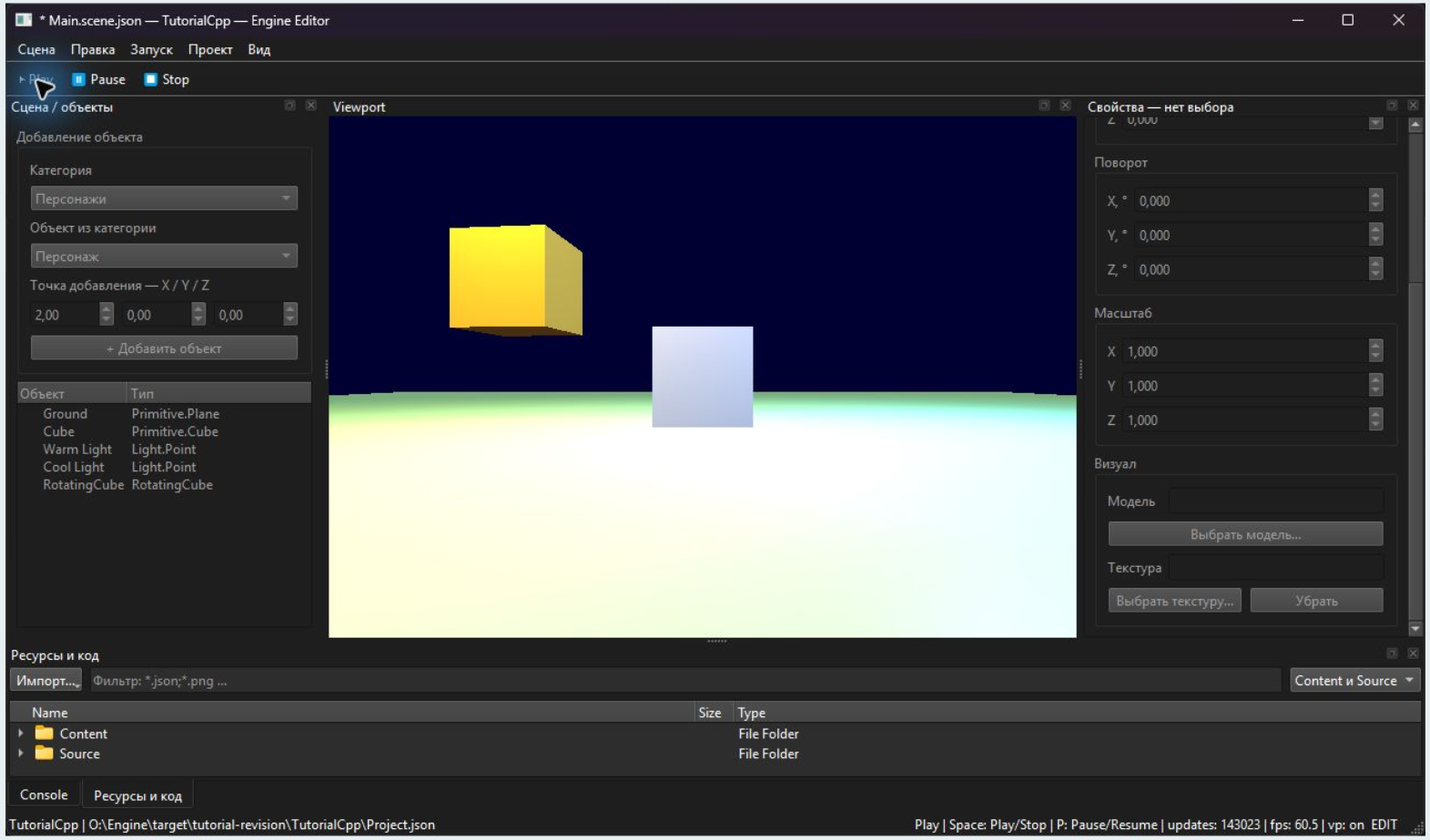
Play запускает копию сцены. Pause / Resume — пауза и продолжение.

Stop

Возвращает рабочую сцену и камеру Edit.

Отдельное окно

Отдельный запуск использует сохранённые файлы проекта.



В viewport: Space — Play / Stop, P — Pause / Resume.

Сохранение и структура проекта

Файл / папка	Назначение
Project.json	Пути к модулю, Content и стартовой сцене
Engine.lock.json	Закреплённая версия и конфигурация движка
Content/Scenes	Сцены с объектами, трансформациями и настройками Play
Content/Models, Textures, Materials	Модели, изображения, файлы .mat и .hlsl
Source и Build.ps1	Код и запуск сборки C++ проекта
Binaries	Собранные версии DLL проекта

Ctrl+S сохраняет сцену. Стартовая сцена назначается отдельно: Сцена → Сделать стартовой сценой.

Восстановление и передача игры

После сбоя

При открытии восстановите предложенную копию сцены. Затем сохраните её через «Сохранить как…».

Перед экспортом

Назначьте стартовую сцену и сохраните материалы. Выберите «Экспорт игры…».

Передать папку

Укажите новую папку вне проекта. Запуск у получателя — через Start.cmd.

Снимок сцены создаётся каждые 30 секунд. Экспорт доступен из Release-редактора.

Граница между редактором и C++

Задача	Без кода	Через SDK / C++
Сцена	Модели, свет, материалы	Свои компоненты и данные
Персонаж	Выбор для Play	Свой Character
Камера Play	Слежение, камера сцены	Свой UpdateCamera
Управление	WASD и ПКМ	Действия и обработчики
Свойства	Значения полей	Описание полей в C++
Игровой HUD	Внешний редактор .ui	UIComponent и OnClick

Контроллер создаётся игрой и не является отдельным добавляемым объектом сцены.

Создание C++ класса

Мастер

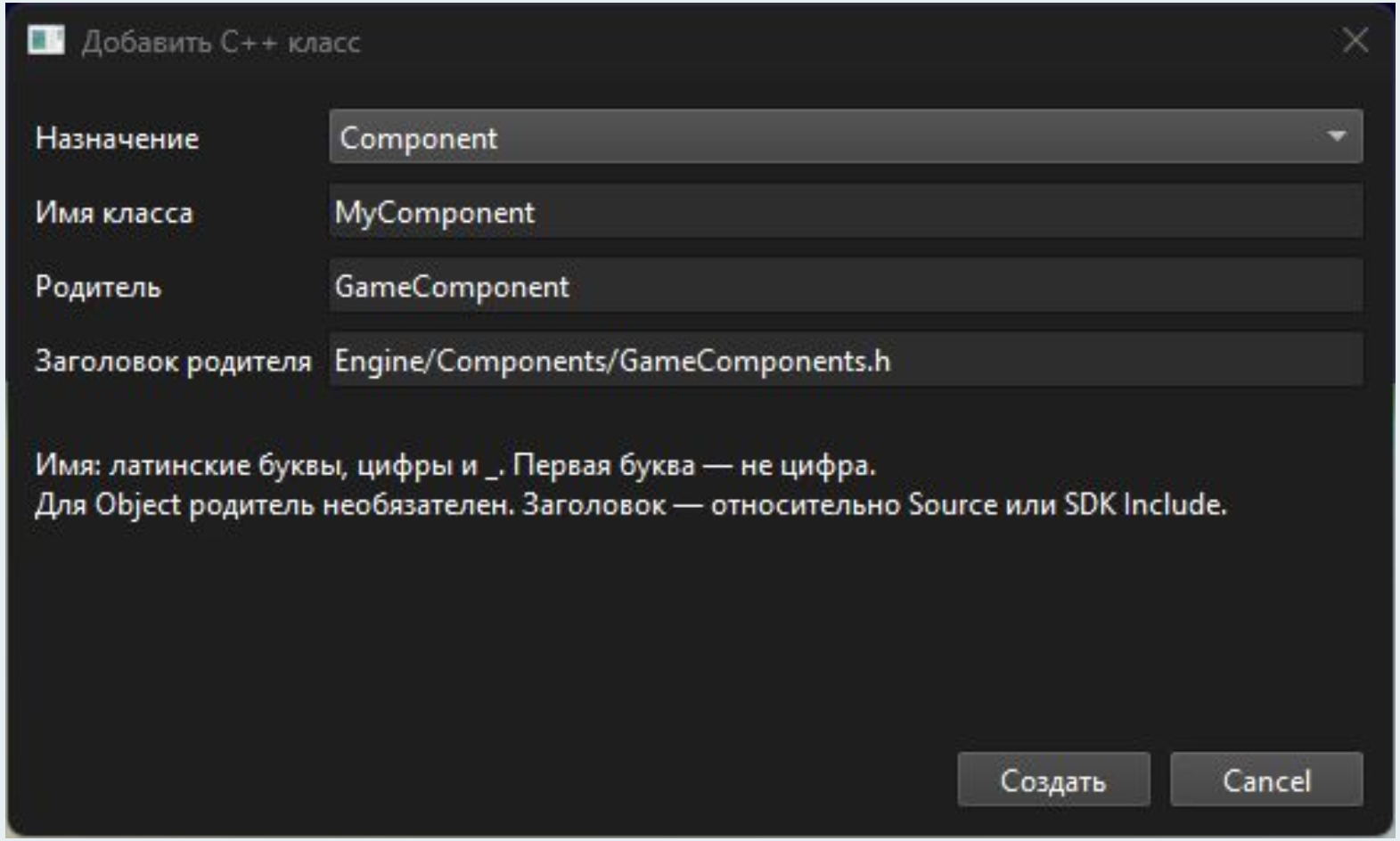
Проект → C++ → Добавить C++ класс… Выберите родителя и задайте имя.

Исходники

Мастер создаёт Source/Имя.h и Source/Имя.cpp.

Сборка

Ctrl+Alt+R — сборка и Hot Reload. Выполняйте в Edit.



Для имени класса: латиница, цифры и _. Начинайте с буквы или _.

Базовый C++ проект: вращающийся куб

Готовый пример

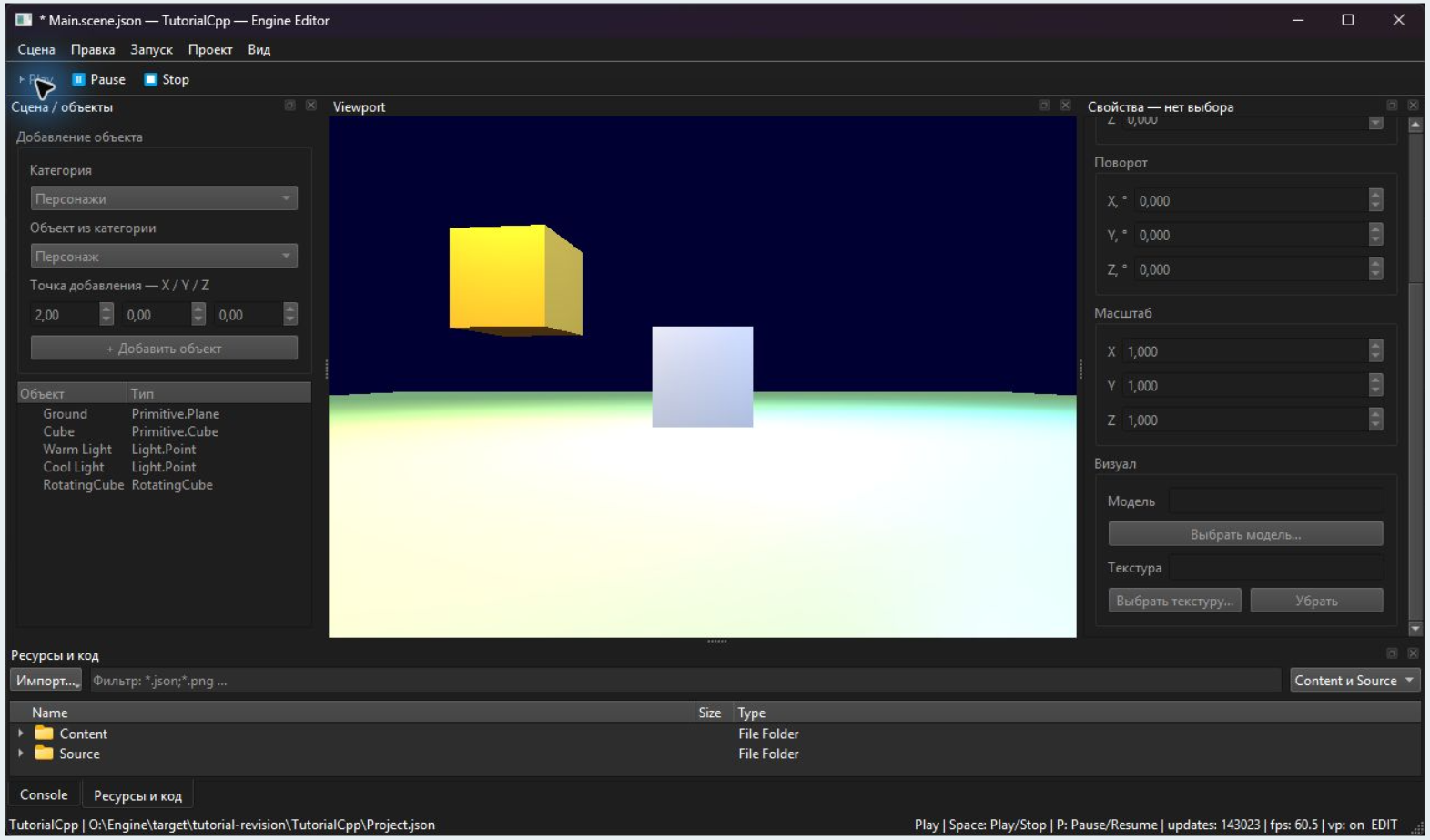
RotatingCube.h/.cpp создаются вместе с C++ проектом.

Запуск

ProjectGame добавляет куб при Play или отдельном запуске.

В Edit

Выберите «Классы проекта» → RotatingCube и добавьте объект.



По умолчанию 90°/с; новый шаблон даёт поля Speed и Enabled в Behavior → Rotation.

RotatingCube: минимальный класс

```
// Source/RotatingCube.h
#pragma once
#include <Engine/Components/ComponentAnnotations.h>
#include <Engine/Components/SpecificComponents/CubeComponent.h>

ENGINE_COMPONENT("RotatingCube")
class RotatingCube : public CubeComponent {
public:
    RotatingCube();
    void Tick(float deltaTime) override;
};

// Source/RotatingCube.cpp
#include "RotatingCube.h"
RotatingCube::RotatingCube() {
    InitCube();
    SetColor({1.0f, 0.65f, 0.2f, 1.0f});
}
```

ENGINE_COMPONENT добавляет класс в каталог проекта.

InitCube создаёт геометрию.

Конструктор задаёт исходный вид.

Вращение на месте в Tick

```
#include "RotatingCube.h"
#include <cmath>

void RotatingCube::Tick(float deltaTime)
{
    CubeComponent::Tick(deltaTime);
    auto rotation = GetRotation();
    rotation.y = std::fmod(
        rotation.y + 90.0f * deltaTime, 360.0f);
    SetRotation(rotation);
}
```

SetRotation принимает градусы.

deltaTime — время шага в секундах.

Позицию не меняем, поэтому куб вращается на месте.

В ProjectGame: SetSimulationTimeScale(1.0f). Полный оборот — четыре секунды.

Практика: два куба, один класс

Открыть проект

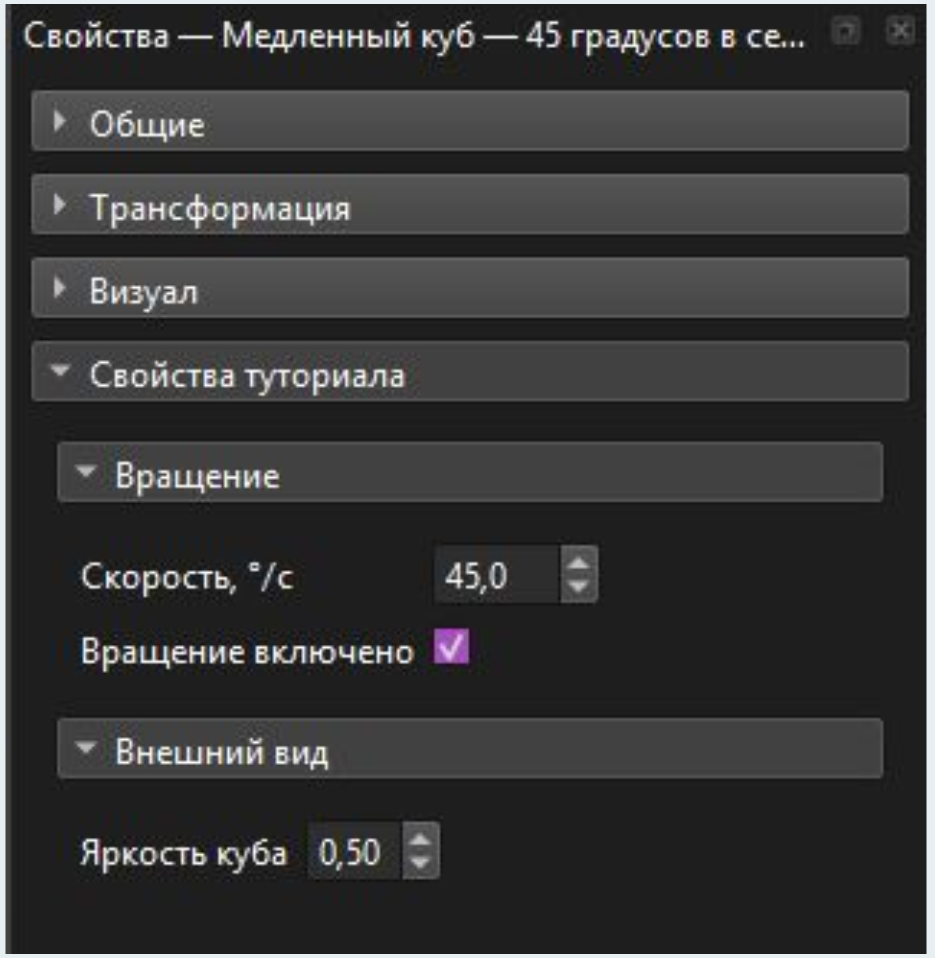
Папка «Туториал-Свойства» → «Открыть редактор.cmd».

Найти поля

Выберите «Медленный куб…». Раскройте «Свойства туториала».

Изменить скорость

В разделе «Вращение»: 45 → 360, Enter. Play покажет результат.



Готовый проект: O:/Engine/Проекты/Туториал-Свойства. Каждый куб хранит свои значения.

Проверьте пользовательские свойства

Действие в Edit	Ожидаемый результат
Скорость: 45 → 360	В Play полный оборот за одну секунду
Скорость: отрицательное число	В следующем Play обратное направление
Снять «Вращение включено»	В Play выбранный куб стоит
Яркость куба: 1 → 0,2	Цвет меняется сразу, без Play
Отменить / Повторить	Возвращается прежнее / новое значение
Сохранить и открыть сцену	Сохраняются отдельные настройки обоих кубов

После проверки нажмите Stop. Для изменения уже объявленного поля пересборка не нужна.

Как поле появляется в инспекторе

```
// RotatingCube.h: дополнительный интерфейс
#include <Engine/Components/EditorProperties.h>
class RotatingCube : public CubeComponent,
                     public EditorPropertyProvider {
    // Остальные объявления — в полном примере.
};

// В DescribeEditorProperties(): один элемент fields
{ "path": "/speed", "type": "number",
  "label": "Скорость, °/с",
  "group": "Свойства туториала/Вращение",
  "min": -720, "max": 720, "step": 15,
  "decimals": 1 }
```

path связывает поле с ключом JSON.

group задаёт вложенные сворачиваемые разделы.

Схема возвращается строкой JSON: version: 1 и массив fields.

Фрагменты сокращены. Полный класс: Game/PropertyTutorial/Source/RotatingCube.h/.cpp.

Своё поле: данные → вид → поведение

Хранить и сохранять	Добавьте поле класса. SaveProjectData записывает его; LoadProjectData читает и проверяет значения.
Описать интерфейс	Добавьте элемент fields в DescribeEditorProperties: path, тип, название и группу.
Использовать значение	В Tick меняйте поведение; в LoadProjectData — внешний вид. Затем соберите и проверьте Undo, Save и Play.

В примере speed_ и enabled_ используются в Tick, brightness_ применяется к цвету сразу.

Создание объекта в мире

```
// В ProjectGame::AfterInitialize()
Game::AfterInitialize();
SetSimulationTimeScale(1.0f);

auto& scene = GetScene();
std::unique_ptr<GameComponent> cube =
    std::make_unique<RotatingCube>();
cube->SetPosition({2.0f, 1.0f, 0.0f});

const auto added = scene.AddComponent(
    "MyCube", cube, "RotatingCube");
if (added.result != SceneResult::Ok &&
    added.result != SceneResult::Deferred)
    throw std::runtime_error("Cannot spawn cube");
```

Подключите RotatingCube.h и <stdexcept>.

Сцена получает владение объектом.

Имя должно быть уникальным. Во время обновления добавление может быть отложено.

Полный ProjectGame.cpp дополнительно проверяет, что куб ещё не добавлен.

Импортированная модель из C++

```
#include <Engine/Scene/SceneAssets.h>

// Файл уже импортирован через редактор.
auto model = SceneAssets::CreateModel(
    "/Game/Models/blend-packed/blend-packed.glb");
model->SetPosition({-2.0f, 0.0f, 0.0f});

const auto added = game.GetScene().AddComponent(
    "MyModel", model, SceneAssets::ModelType);
if (added.result != SceneResult::Ok &&
    added.result != SceneResult::Deferred)
    throw std::runtime_error("Cannot spawn model");
```

/Game — папка Content проекта.

CreateModel читает .import.json и назначает материал, созданный при импорте FBX/BLEND.

Подставьте путь вашей модели.

Полная функция в ModelExample.h/.cpp: SpawnModel(game, "MyModel", путь_модели).

Материал и текстура из C++

```
// object — GameComponent&, ресурс уже в Content.
const std::string material = "/Game/Materials/Wood.mat";
SceneAssets::ValidateMaterial(material);
if (!object.SetMaterialAsset(material))
    throw std::runtime_error("Material unsupported");

// Отдельная текстура объекта: материал сохраняется.
if (!object.TrySetTexture("/Game/Textures/grid.png"))
    throw std::runtime_error("Cannot set texture");

// Убрать отдельную текстуру, оставив материал:
object.TrySetTexture("");
```

BaseMap меняйте в редакторе материала.

Отдельная текстура объекта дополнительно умножает его цвет.

SetMaterialAsset назначает ссылку; шейдер компилируется при отрисовке.

Для новой геометрии используйте CreateModel. TrySetTexture меняет изображение, а не модель.

Ввод: поля и жизненный цикл

```
#pragma once
// Source/InputCube.h
#include "RotatingCube.h"
#include <Engine/InputDevice/InputActions.h>

ENGINE_COMPONENT("InputCube")
class InputCube : public RotatingCube {
public:
    void BeginPlay() override;
    void Tick(float deltaTime) override;
private:
    bool spinning_ = true;
    unsigned nextCube_ = 1;
    Input::Connection toggle_, spawn_;
    Input::Scope inputScope_;
    void SpawnCube();
};
```

BeginPlay вызывается перед первым игровым вводом и Tick.

Connection нужно хранить.

Scope объявлен последним из полей: он отключит обработчики до разрушения их состояния.

Скопируйте InputCube.h/.cpp из примеров в Source и добавьте один InputCube в сцену после сборки.

Клавиша R: остановить вращение

```
void InputCube::BeginPlay()
{
    RotatingCube::BeginPlay();
    auto& input = GetGame()->GetInput();
    input.Bind({"ToggleRotation",
        Input::Context::Game, {Keys::R}});
    toggle_ = input.Subscribe(inputScope_,
        Input::Context::Game, "ToggleRotation",
        Input::Phase::Pressed,
        [this](const Input::Event&) {
            spinning_ = !spinning_;
        });
    // Здесь же добавьте привязку N со следующего слайда.
}
```

R переключает вращение один раз на нажатие.

В Tick: RotatingCube::Tick(spinning_ ? deltaTime : 0.0f);

После Play щёлкните viewport.

Контекст Game получает ввод в сфокусированном viewport в Play; Pause и потеря фокуса сбрасывают действия.

Клавиша N: создать новый куб

```
// Продолжение InputCube::BeginPlay()
input.Bind({"SpawnCube",
    Input::Context::Game, {Keys::N}});
spawn_ = input.Subscribe(inputScope_,
    Input::Context::Game, "SpawnCube",
    Input::Phase::Pressed,
    [this](const Input::Event&) { SpawnCube(); });

// В InputCube::SpawnCube()
std::unique_ptr<GameComponent> cube =
    std::make_unique<RotatingCube>();
cube->SetPosition(GetLocalPosition() +
    glm::vec3(2.0f * number, 0, 0));
const auto added = scene.AddComponent(
    name, cube, "RotatingCube");
```

SpawnCube получает GetGame()->GetScene().

number — счётчик, name — свободное имя SpawnedCubeN.

В полном файле проверяются имя и результат добавления.

Новый куб появляется в копии Play. После Stop он исчезает вместе с этой копией.

Сочетания и переназначение

```
// В BeginPlay после привязки ToggleRotation
input.Unbind(Input::Context::Game, "ToggleRotation");
input.Bind({"ToggleRotation", Input::Context::Game,
            {Keys::Control, Keys::R}});

// Две привязки одной оси
input.Bind({"MoveX", Input::Context::Game,
            {Keys::D}, 1});
input.Bind({"MoveX", Input::Context::Game,
            {Keys::A}, -1});
```

Обработчик ToggleRotation остаётся прежним.

Ctrl + R — одновременное удержание клавиш.

Для MoveX подпишитесь на Phase::Axis и читайте e.value. A + D даёт ноль.

Своя схема управления

```
class MyController : public CharacterController {
public:
    using CharacterController::CharacterController;
    void BindInput() override {
        game_.GetInput().Bind({"Go",
            Input::Context::Game, {Keys::I}});
        connections_.push_back(game_.GetInput().Subscribe(
            inputScope_, Input::Context::Game, "Go",
            Input::Phase::Axis,
            [this](const Input::Event& e) {
                forward_ = e.value;
            }));
    }
};
```

I задаёт движение вперёд и заменяет стандартные привязки.

Game выбирает контроллер в методе фабрики. Пример — в README.txt.

Для своей камеры переопределите UpdateCamera().

Выберите MyGame: Проект → C++ → Типы C++ проекта…; персонажа назначьте в настройках Play.

Игровой UI в файле .ui

Разметка

UI и Group собирают дерево. Panel, Label, Bar и Button задают его внешний вид.

Подключение

C++ создаёт UIComponent, загружает файл и назначает действия кнопкам.

Редактирование

Меняйте XML во внешнем редакторе и сохраняйте. Игра обновит уже загруженный HUD.

Отдельного визуального окна UI в редакторе пока нет. HUD рисуется поверх игрового кадра.

Разметка панели HUD

```
<UI>
  <Panel name="stats" rect="16 16 280 144"
         padding="12" color="#101820DD">
    <Label name="clicks" text="Нажатий: 0"
           rect="0 0 240 22"/>
    <Bar name="load" rect="0 32 240 12"
         fill="0" fillColor="#35B8CFFF"/>
    <Button name="add" rect="0 64 240 32"
            text="Добавить"/>
  </Panel>
</UI>
```

Сохраните в Content/UI/Hud.ui, кодировка UTF-8.

name связывает элемент с C++.

padding задаёт отступ для дочерних элементов.

Полный пример: папка UI в архиве исходников туториала.

Размеры и привязки UI

Атрибут	Как работает
rect="x y width height"	Смещение и размер в пикселях внутри родителя
anchor="TopRight"	Отступ от правого верхнего края внутрь панели
anchor="BottomLeft"	Отступ снизу. Для текста задайте высоту явно
width / height = 0	Растянуть до края родительской области
padding="12"	Один отступ или четыре: слева, сверху, справа, снизу
color="#RRGGBBAA"	Цвет и прозрачность. #RRGGBB полностью непрозрачен

Подключение HUD к игре

```
// ProjectGame.cpp
#include "TutorialHud.h"

// После Game::AfterInitialize(), до ранних return:
AttachTutorialHud(*this,
    "C:/Games/Demo/Content/UI/Hud.ui");

// В конструкторе TutorialHud:
FontAtlasSettings font;
font.Face = L"Consolas";
font.PixelHeight = 18;
SetFont(font);
if (!LoadMarkup(markupFile.generic_u8string()))
    throw std::runtime_error("Cannot load HUD");
```

Путь на слайде замените своим.

LoadMarkup читает файл с диска. /Game автоматически не разрешается.

Шрифт задайте до первого кадра.

TutorialHud.h/.cpp содержат регистрацию и проверку ошибок. HUD создаётся только при запуске игры.

После обновления привяжите элементы заново

```
void TutorialHud::BindElements() {
    revision_ = GetMarkupRevision();
    label_ = Find<UILabel>("clicks");
    bar_ = Find<UIBar>("load");
    if (auto* b = Find<UIButton>("add"))
        b->OnClick = [this] { ++clicks_; };
}

void TutorialHud::Tick(float dt) {
    UIComponent::Tick(dt);
    if (GetMarkupRevision() != revision_)
        BindElements();
    if (label_) label_->SetText(
        "Нажатий: " + std::to_string(clicks_));
}
```

Замена дерева делает старые указатели недействительными.

Find может вернуть nullptr.

OnClick тоже назначается заново.

Нажатие и отпускание ЛКМ должны попасть в одну кнопку. Увод мыши отменяет щелчок.

Практика с HUD в Battle

Запустите Battle

HUD показывает время кадра, вызовы отрисовки, количество техники и уровни LOD.

Измените файл

Рядом с Battle.exe откройте GameContent/UI/Hud.ui. Измените color или rect и сохраните.

Проверьте результат

Раскладка обновится. Кнопка и F6 переключают раскраску LOD, F7 скрывает HUD.

Постоянные правки сохраните также в Game/Battle/Content/UI/Hud.ui перед следующей сборкой.

Если HUD не обновился

Симптом	Что проверить
Нет HUD при запуске	Физический путь, UTF-8 и сообщение UIComponent в логе
Старый вид после сохранения	Загруженную копию .ui и строку ошибки XML
Изменение атрибута не видно	Неизвестный атрибут игнорируется с предупреждением
Кнопка без действия	name, Find<UIButton> и OnClick после смены ревизии
Камера реагирует на щелчок	В игровой логике учитывайте GetHovered()
Символы пропали	Атлас содержит ASCII и русские буквы А-я, Ё и ё

Обновление движка и проекта

Обычный запуск

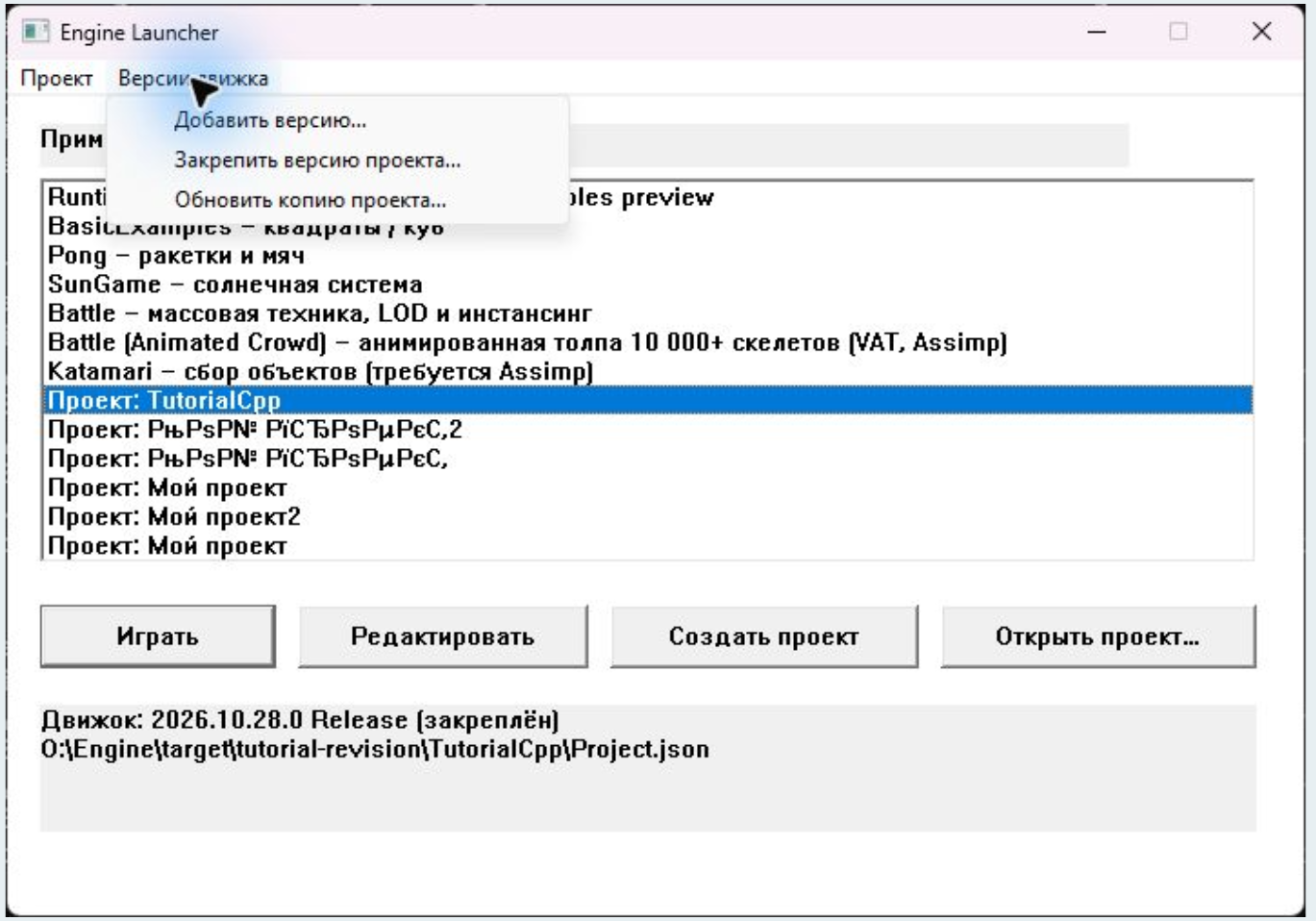
Нажмите «Играть» или «Редактировать» у старого проекта.

Предложение

Лаунчер покажет версии и предложит обновить копию проекта.

Выбор

Можно открыть исходник на прежней установленной версии или отменить.



Обновление сохраняет исходную папку. Меню на снимке — ручной способ: «Обновить копию проекта…».

Частые вопросы при первом запуске

Симптом	Что проверить
Персонаж не двигается	Play, фокус viewport, назначение и флажок управления
BLEND не импортируется	Blender 4.2+, доступные картинки или упакованные текстуры
Материал рисуется шахматкой	Ошибки HLSL, путь .mat; после исправления нажмите F6
Новый класс не виден	Успешная сборка, регистрация и Hot Reload в Edit
Ошибка сборки / загрузки	Console, CodeBuild.log и логи кандидата в Binaries
Пропала модель или текстура	Ресурс импортирован в Content и перенесён вместе с проектом